# Machine-learning Best Practices

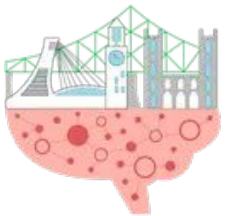
## {Part 1 - Supervised Learning}

QLS 612 | May 2026

*By*

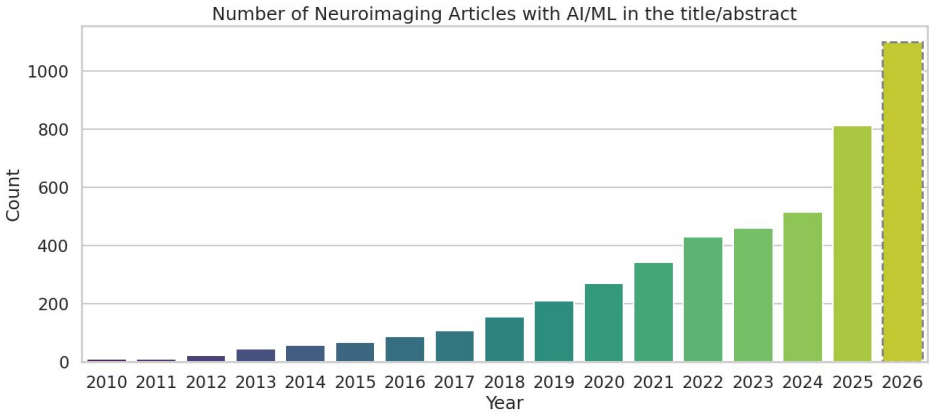
*Nikhil Bhagwat*

*ORIGAMI Lab, McGill*

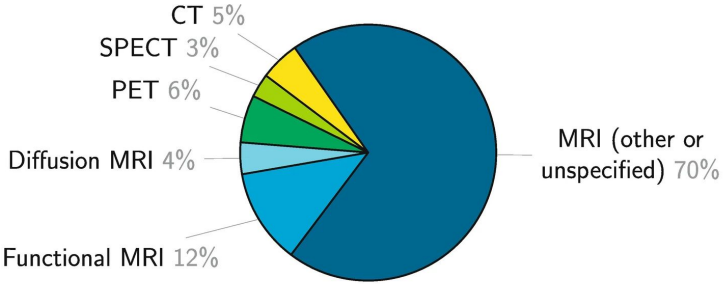


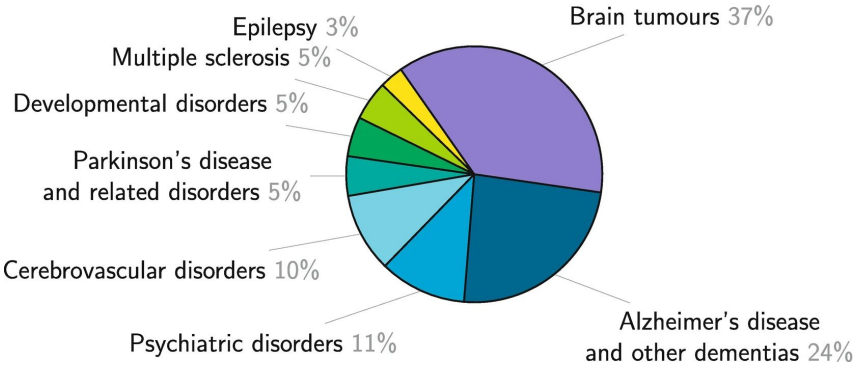
# The Trends

# AI/ML articles in Neuroimaging



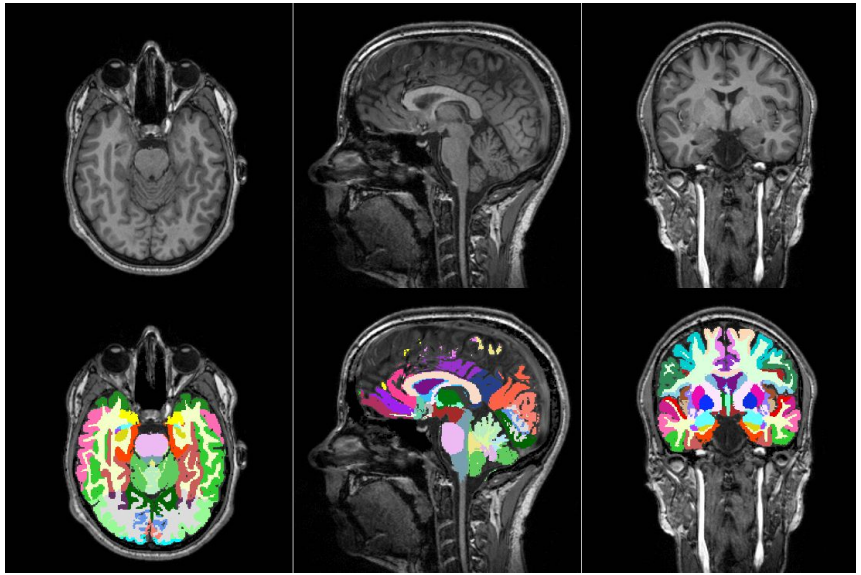

***Pre and post LLM era?***



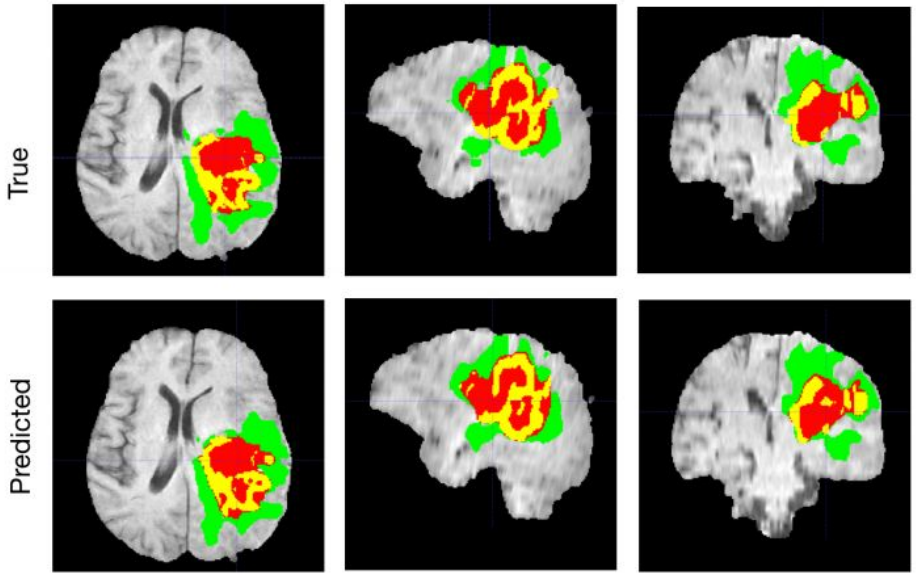




Neuroimaging in Machine Learning for Brain Disorders. (Burgos et al 2023)

# Applications: Segmentation



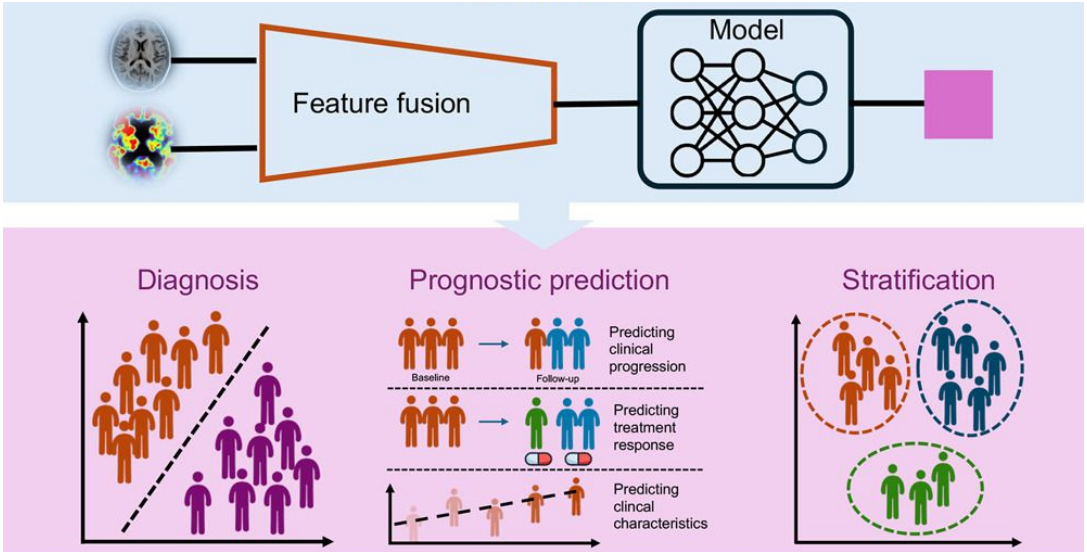
Source: https://github.com/fepegar/torchio




https://developer.nvidia.com/blog/automatically-segmenting-brain-tumors-with-ai/

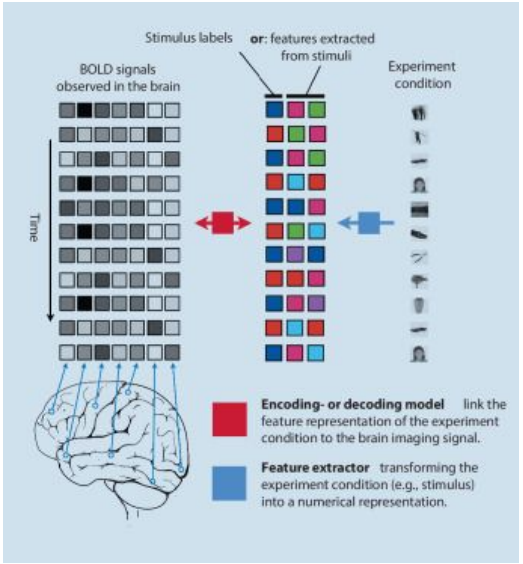
# Applications: Clinical task

**Clinical use-cases**



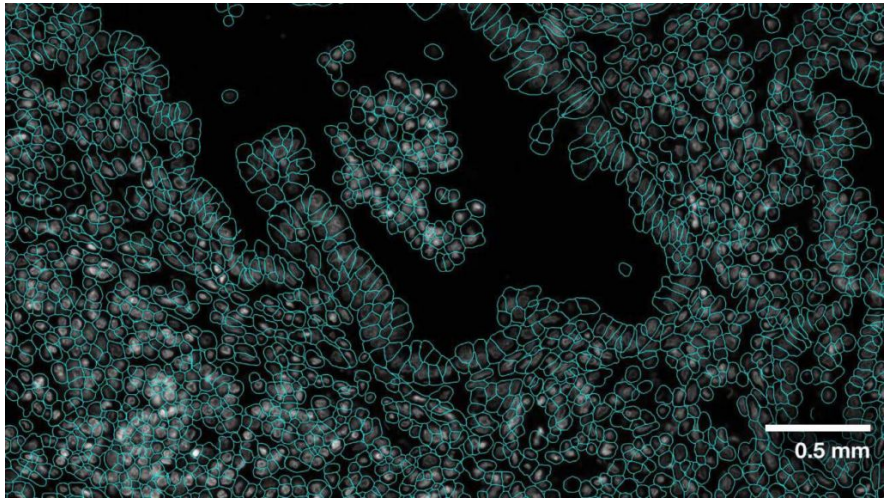

Huang, Weijie et al. 2025. Cell Reports Medicine

**Brain function → Task**

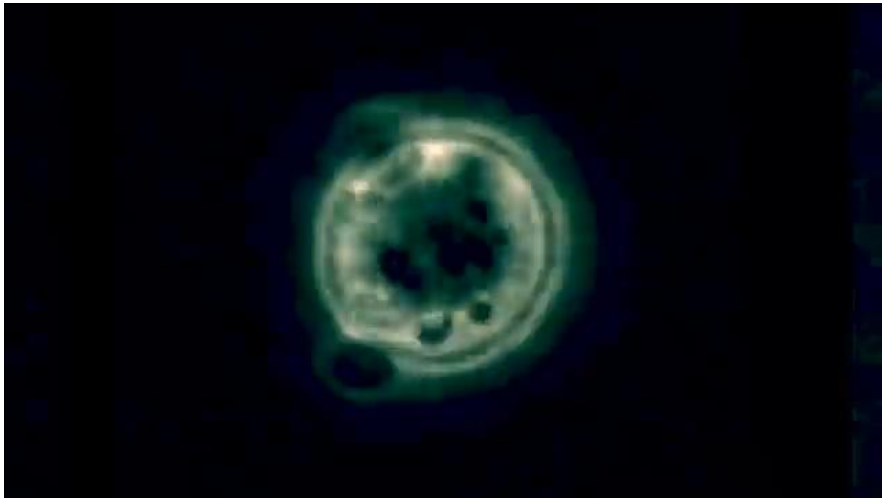


# AI/ML in Cells

Cell segmentation

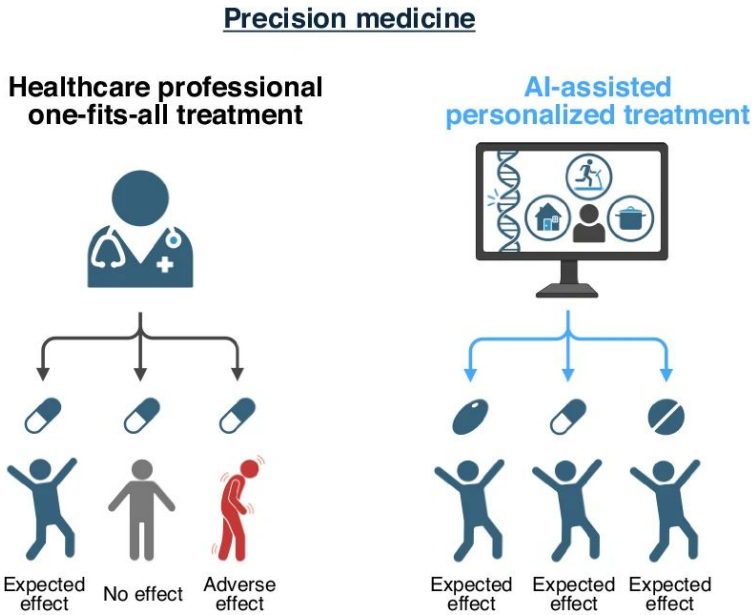



Cancer prediction

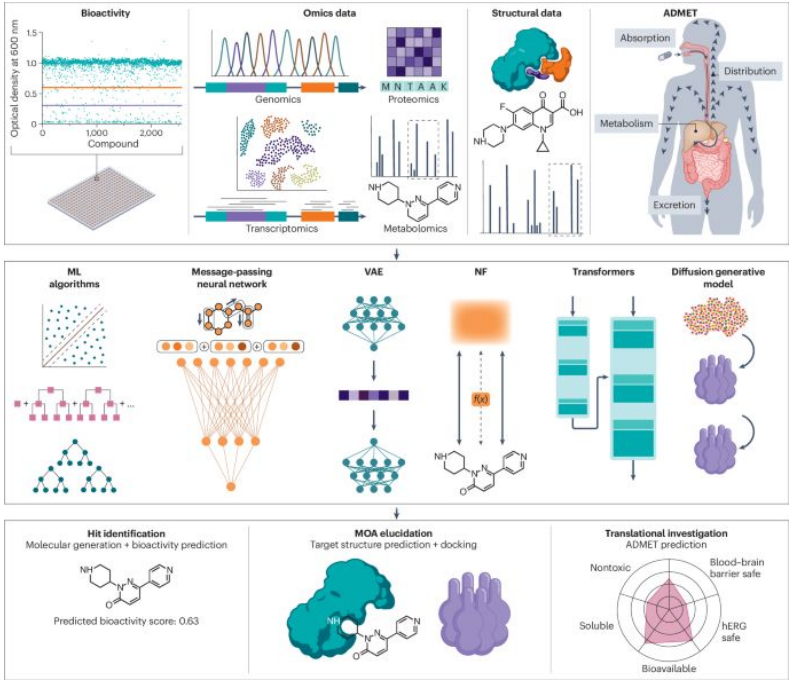


https://developer.nvidia.com/blog/advancing-cell-segmentation-and-morphology-analysis-with-nvidia-ai-foundation-model-vista-2d/

# AI/ML in Medicine



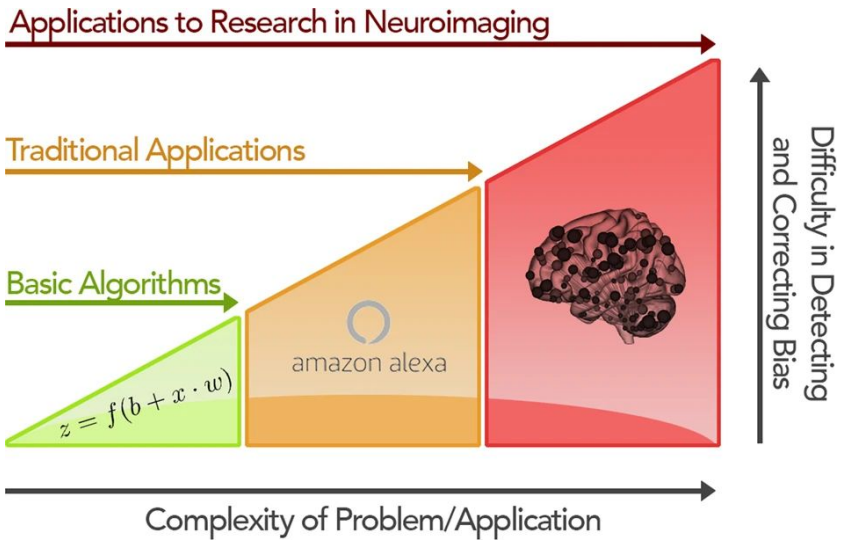

O'Connor, O. et al. 2025. Increasing use of artificial intelligence in genomic medicine for cancer care- the promise and potential pitfalls.




Catacutan, D.B., et al. 2024. Machine learning in preclinical drug discovery. Nat Chem Biol

# Current status of Neuroimaging x ML

- Good news: very useful for image processing tasks
- Bad news: yet to have any meaningful clinical use case → poor replication

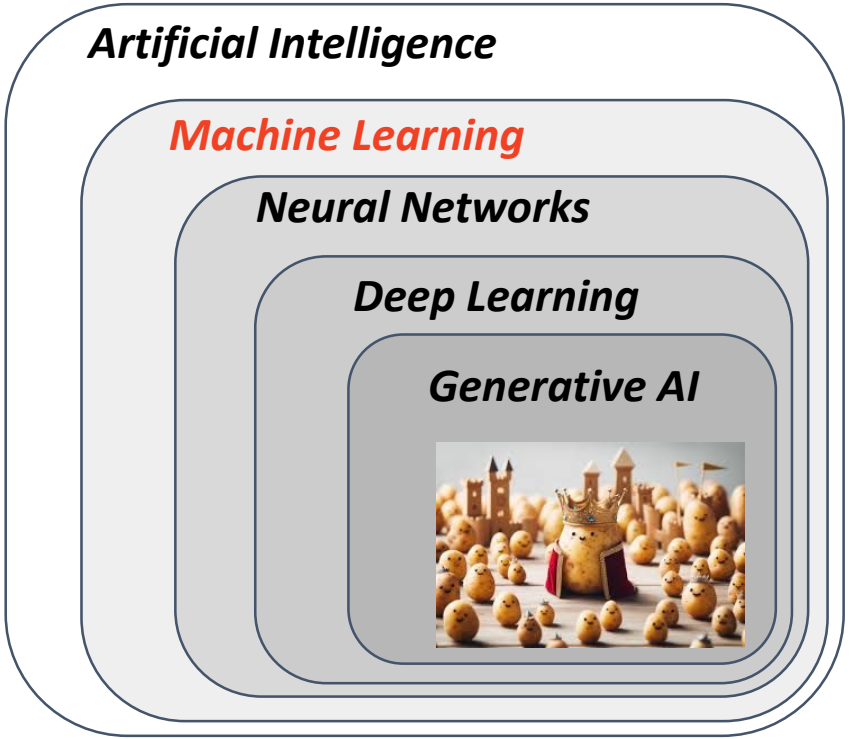



Tejavibulya, L., Rolison, M., Gao, S. *et al.* Predicting the future of neuroimaging predictive models in mental health. *Mol Psychiatry* (2022).

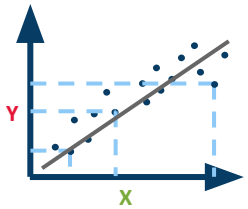
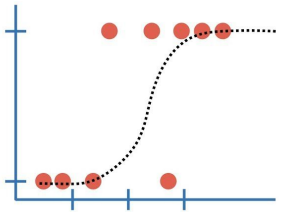
# The Basic Terminology

# What is Machine-learning

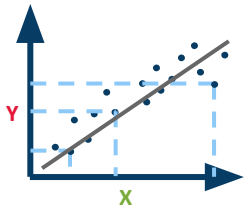
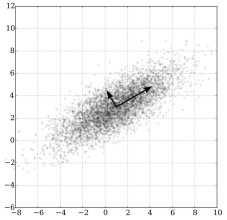
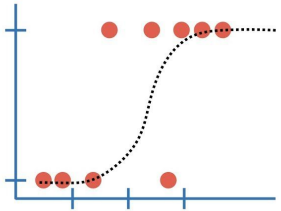
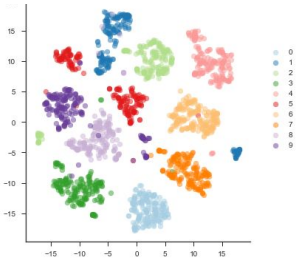
ML is the study of computer algorithms that improve automatically through experience and by the use of data.



# Types of ML Algorithms

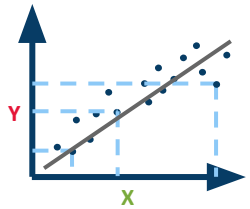
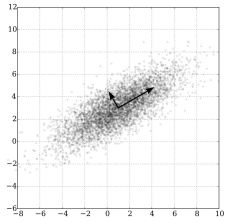
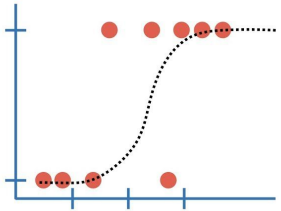
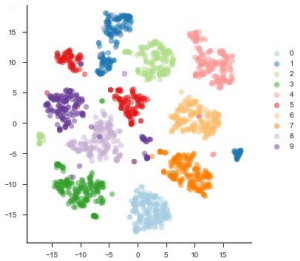
| Outcome | Supervised Learning | |
|---|---|---|
| Continuous | Regression   | |
| Categorical | Classification  | |

# Types of ML Algorithms

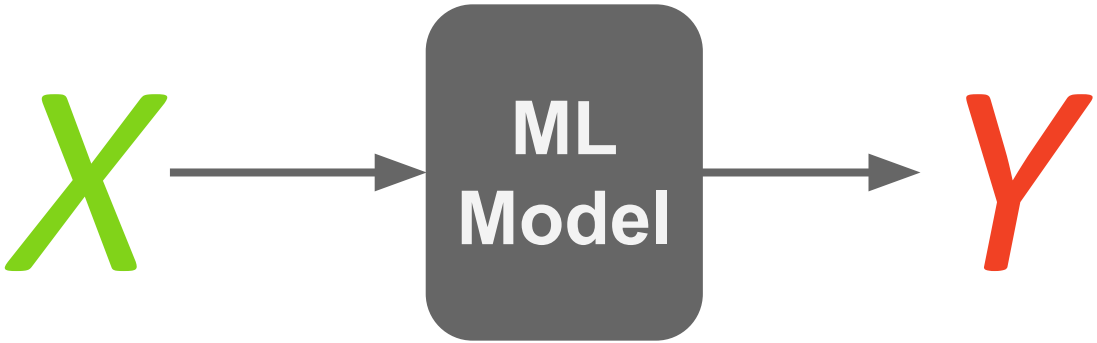
| Outcome | Supervised Learning | Unsupervised Learning |
|---|---|---|
| **Continuous** | Regression   | Principal Component  |
| **Categorical** | Classification  | Clustering  |

** … and Reinforcement Learning*

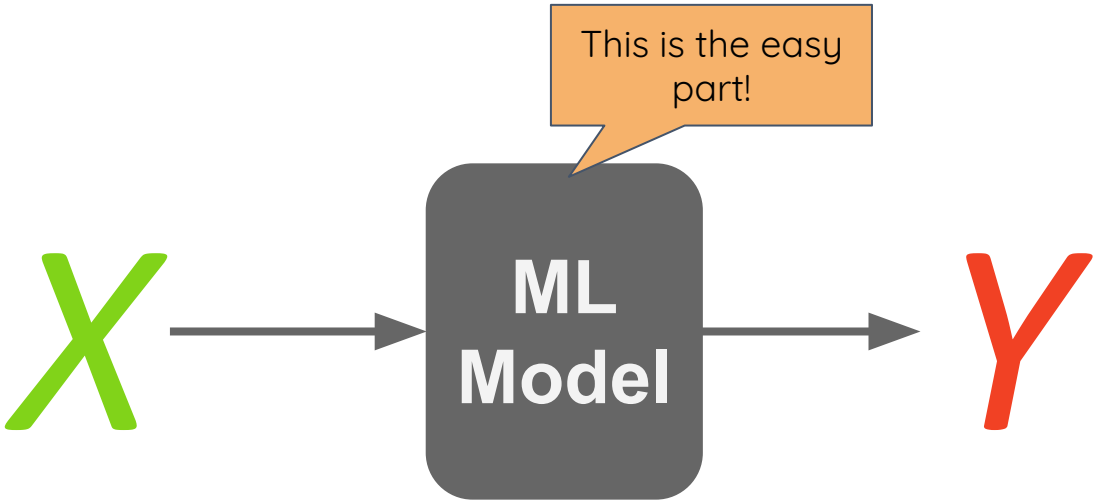
# Types of ML Algorithms

| Outcome | Supervised Learning | Unsupervised Learning |
|---|---|---|
| Continuous | Regression   | Principal Component  |
| Categorical | Classification  | Clustering  |

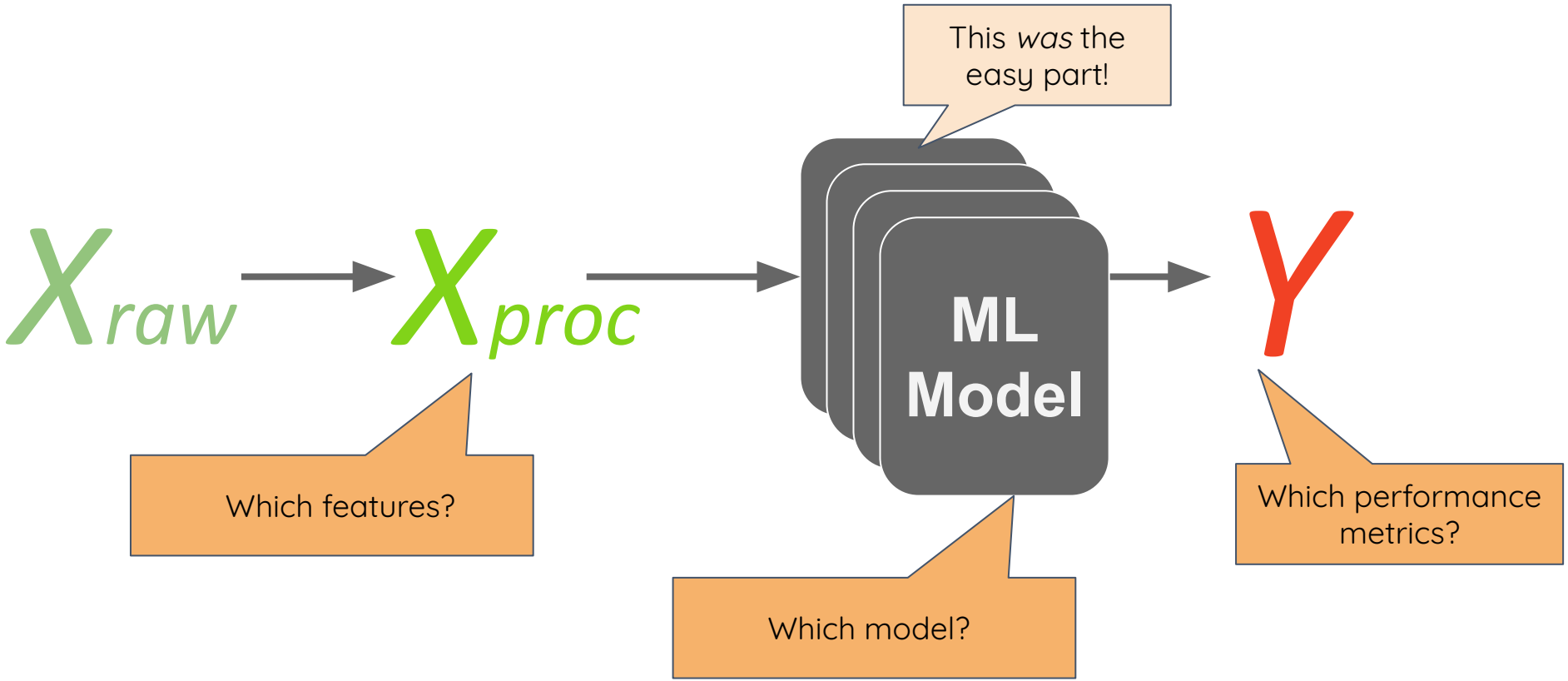
# Training a machine-learning model



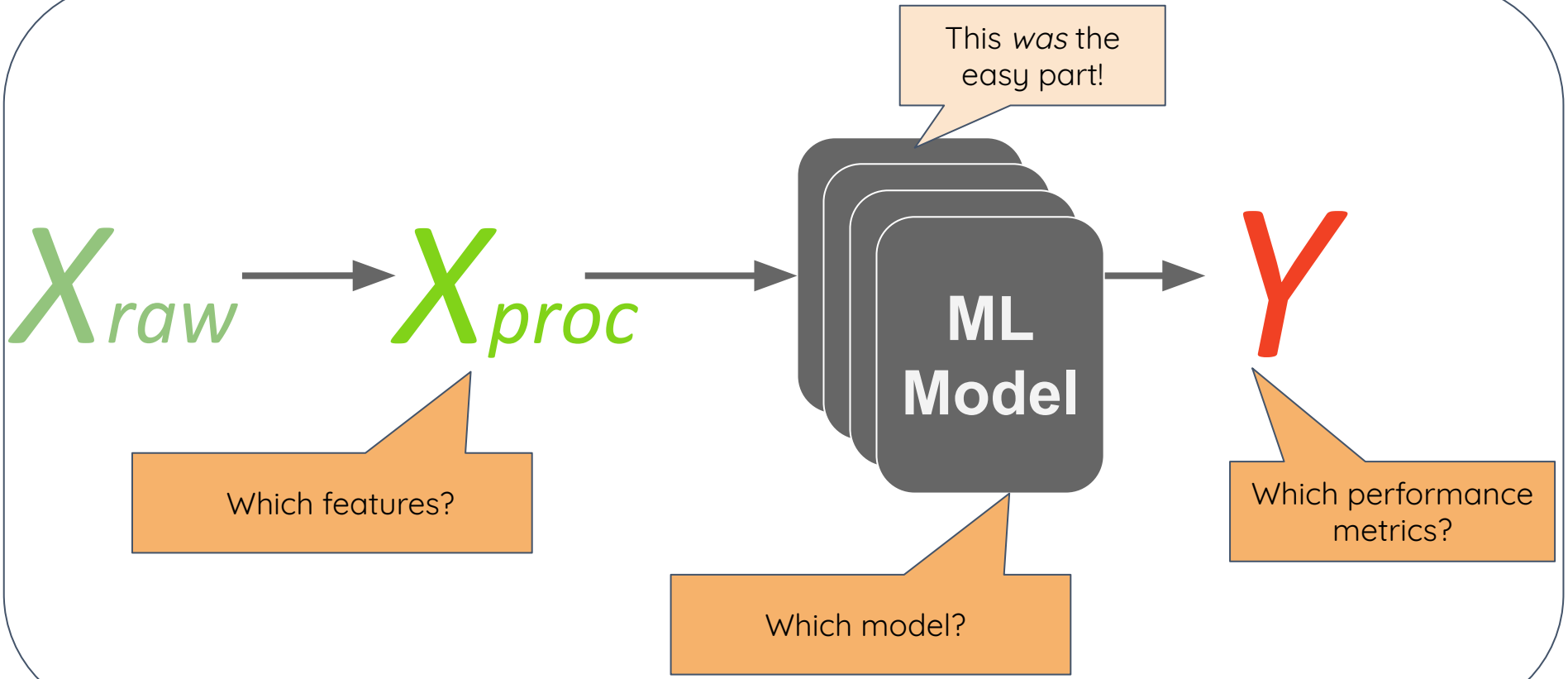
# Training a machine-learning model



# Training a ML model in Neuroimaging



How do I validate my design choices?



# Pop Quiz

# Pop Quiz

Goal: Diagnose Schizophrenia (SZ)

Data: {MRI, demographics, psychiatric assessments, diagnostic labels}

We start with splitting data 80% (train) and 20% (test).

We train a model using MRI and we have 91% accuracy on the test set.

*How happy should we be?*

a) Very: SZ is solved - wait for the Nobel prize!

b) Somewhat: This is an interesting result we can publish so others can replicate.

c) A little: 91% is a good start! But we need to do more checks.

d) Not at all

# Pop Quiz

Goal: Diagnose Schizophrenia (SZ)

Data: {MRI, demographics, psychiatric assessments, diagnostic labels}

**Lit search: Our population has 1% SZ prevalence**

We start with splitting data 80% (train) and 20% (test).

We train a model using MRI and we have 91% accuracy on the test set. **Then we also calculate,**

- **90% sensitivity (i.e. probability that prediction is positive if patient has SZ)**
- **91% specificity (i.e. probability that prediction is negative if patient doesn't have SZ)**

The clinician asks you what are the chances that my patient has SZ, if your model says SZ?

A) 9 in 10 B) 1 in 2 C) 1 in 10 D) 1 in 100

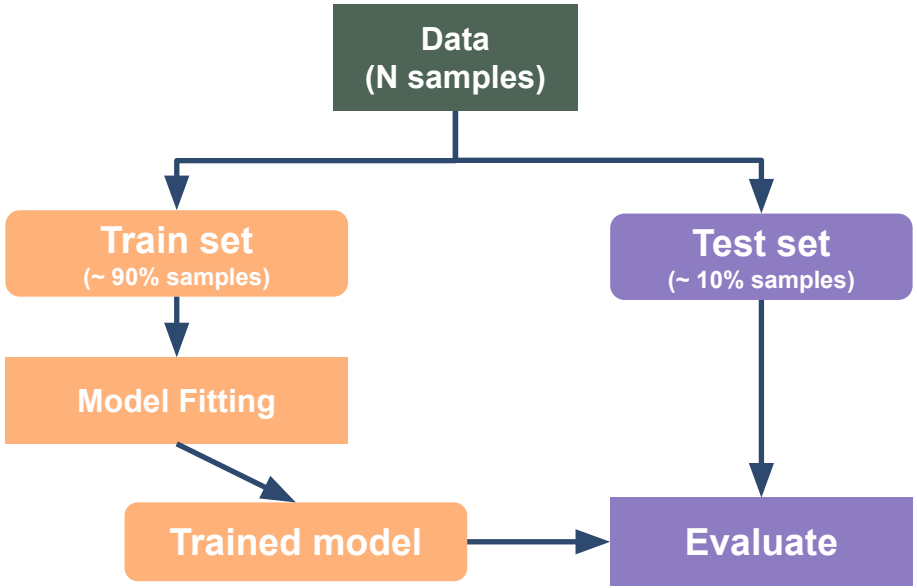
*Also how happy should we be now?*

# What is a good model?

# Model Evaluation

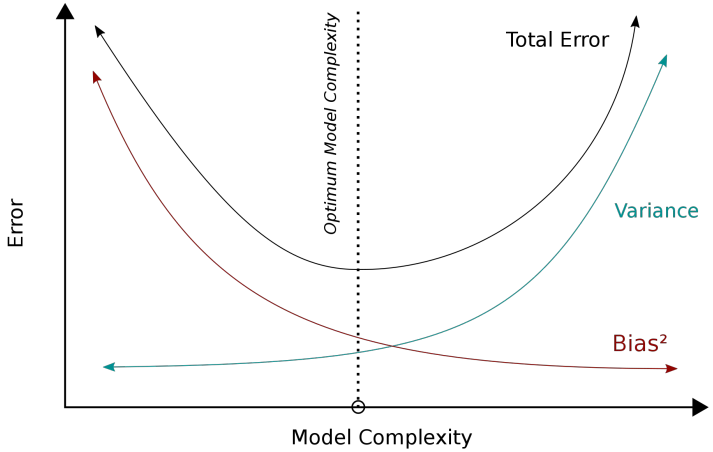
- Is the model learning something useful?
  - Overfitting vs underfitting
- Is the model generalizable?
  - Same dataset vs different datasets
- Is the model reliable?
  - Robustness, certainty, explainability

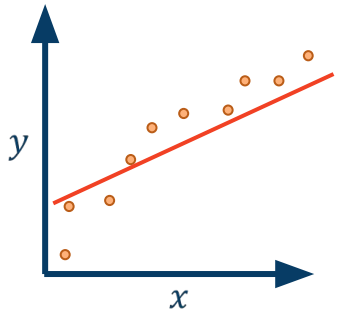
# Measuring model performance



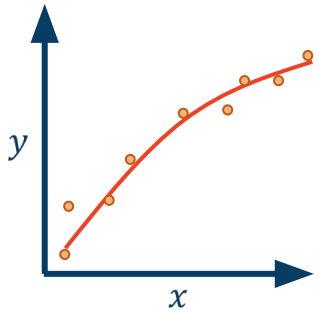
# Underfitting vs Overfitting

- Train performance ≠ Test performance
- Errors: Bias - Variance tradeoff
- Regression example

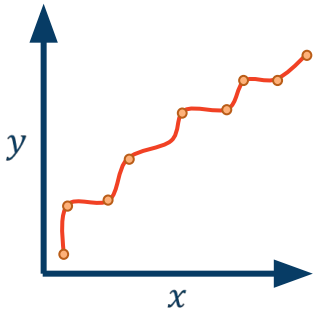



Underfitting




Optimal

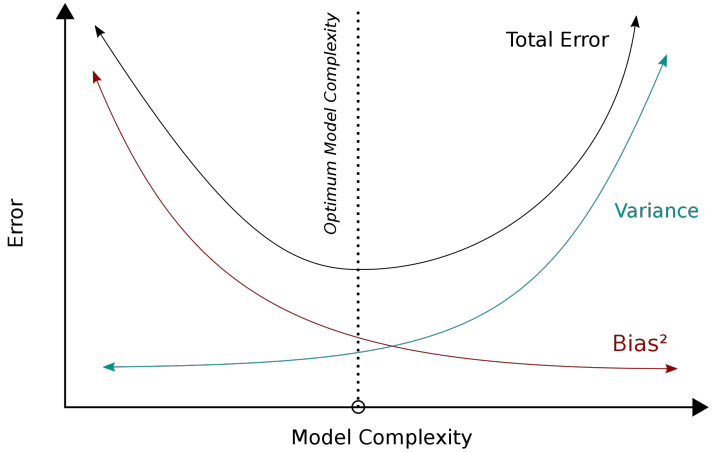


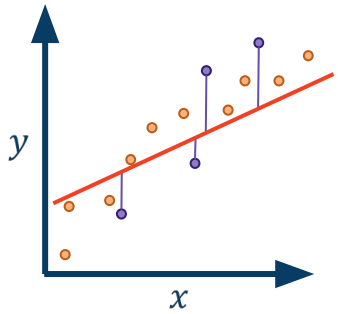

Overfitting

Train set

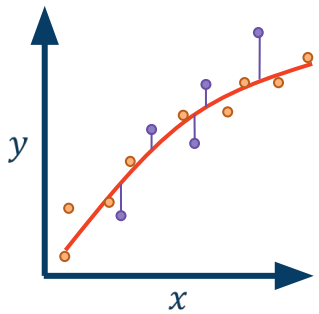
# Underfitting vs Overfitting

- Train performance ≠ Test performance
- Errors: Bias - Variance tradeoff
- Regression example

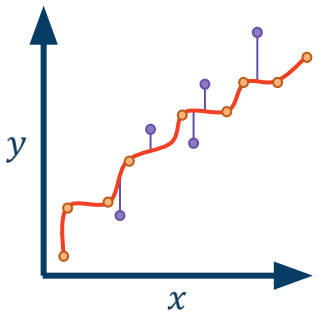



Underfitting




Optimal



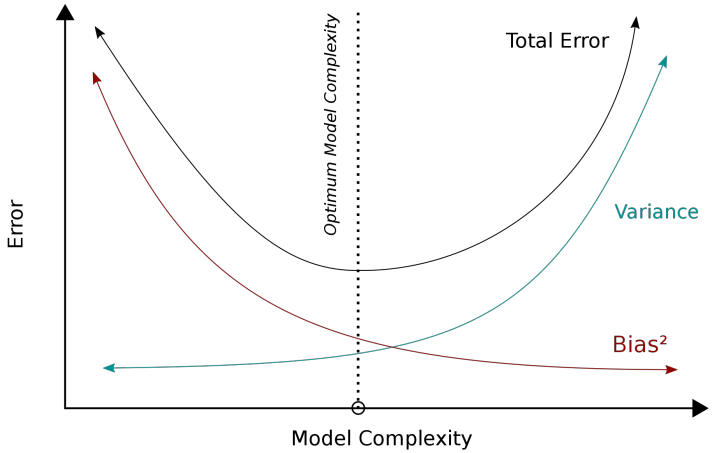

Overfitting

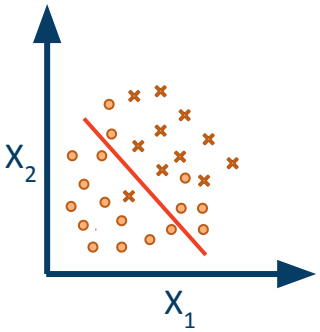
Train set

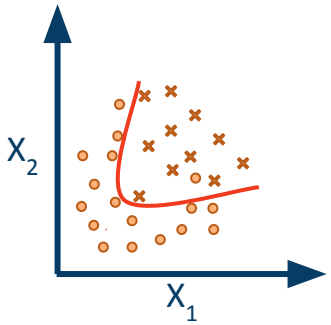
Test set

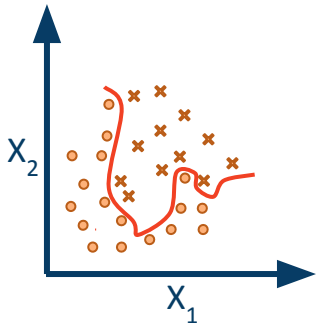
# Underfitting vs Overfitting

- Train performance ≠ Test performance
- Errors: Bias - Variance tradeoff
- Classification example



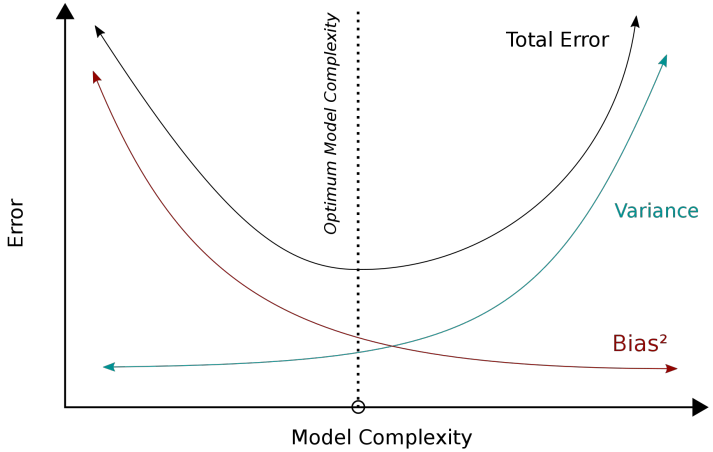



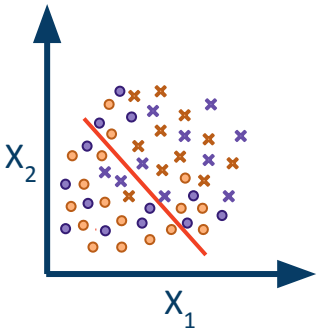



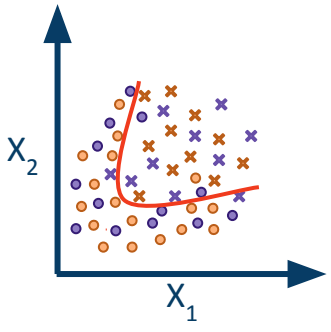

Train class_1 Train class_2

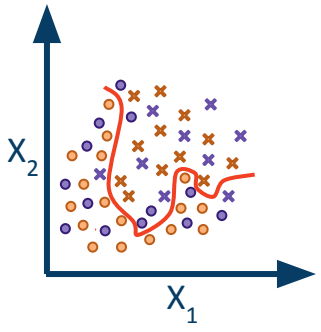
# Underfitting vs Overfitting

- Train performance ≠ Test performance
- Errors: Bias - Variance tradeoff
- Classification example



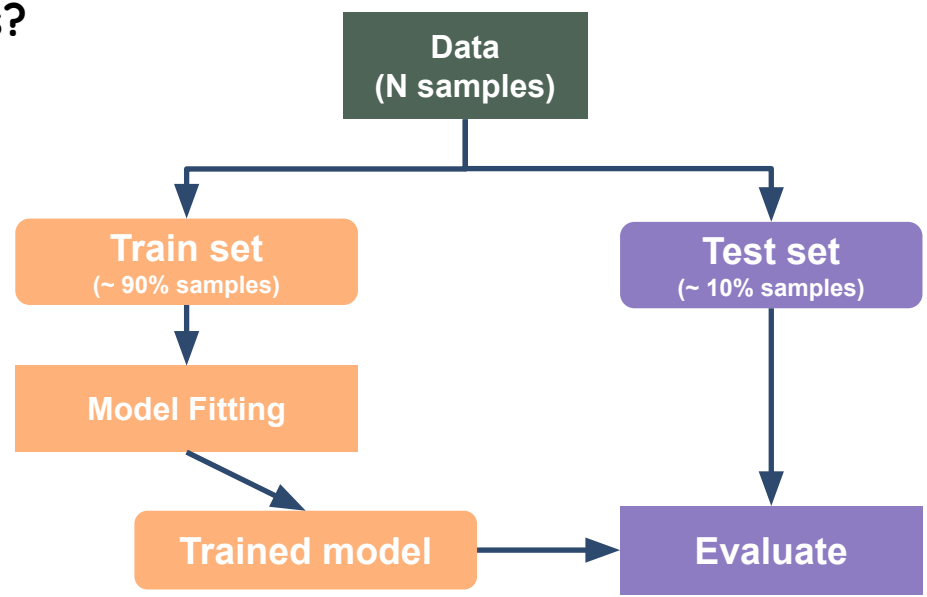




Train class_1 Train class_2 Test class_1 Test class_2

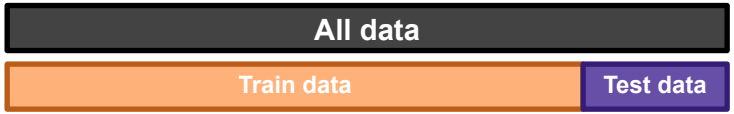
# Measuring model generalizability

- **How do we sample train and test sets?**
- How do we select a model?



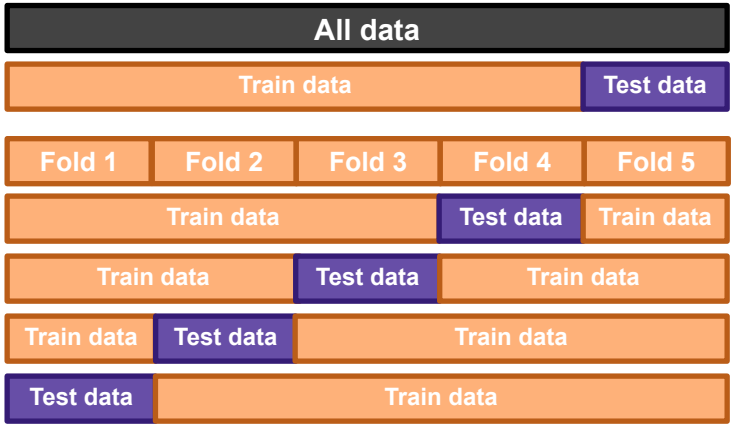
# Cross-validation (outer loop)

- How do we sample train and test sets?
  - Train set: learn model parameters
  - Test set (a.k.a held-out sample): Evaluate model performance



# Cross-validation (outer loop)

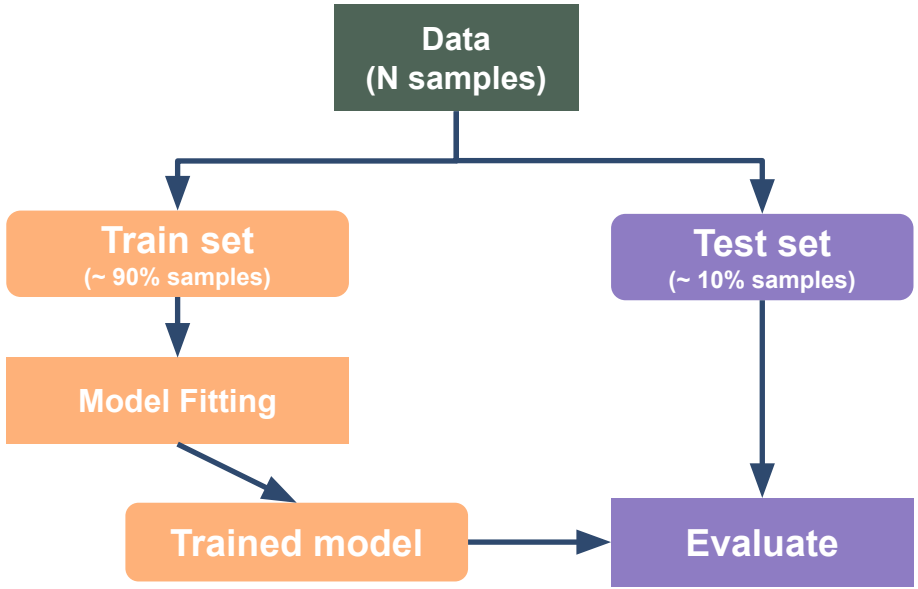
- How do we sample train and test sets?
  - Train set: learn model parameters
  - Test set (a.k.a held-out sample): Evaluate model performance
  - Repeat for different Train-Test splits
    - k-fold, shuffle-split
  - Report performance statistics over all test folds




CV outer loop

# Measuring model generalizability

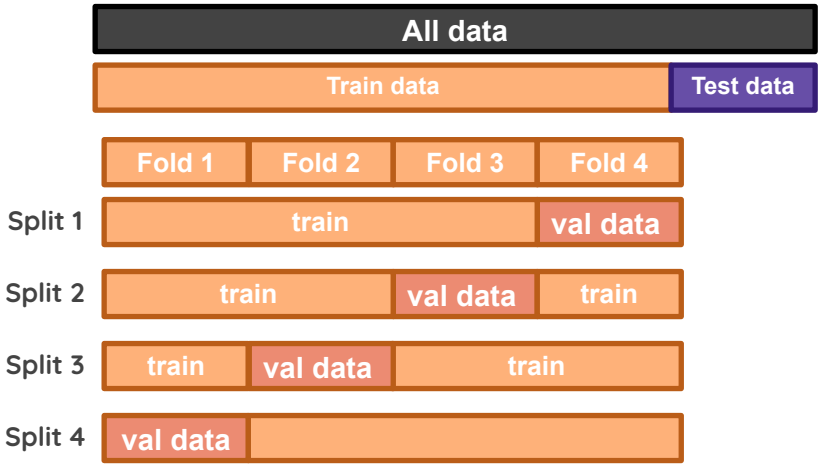
- How do we sample train and test sets?
- **How do we select a model?**
  - SVM, XGBoost, CNNs
  - Hyper-parameters



# Cross-validation (inner loop)

- How do we select a model?
  - Tune *hyper-parameters* of a model
  - Compare several different model architectures
  - Select / transform raw features

- This repeats for all train-test splits in the outer loop




CV inner loop

# Hyper-parameters

- Hyper-parameter ≠ parameter (or weights)
  - Parameters are **learned**; hyper-parameters are **chosen**!

# Hyper-parameters

- Hyper-parameter ≠ parameter (or weights)
  - Parameters are **learned**; hyper-parameters are **chosen**!

- Examples:
  - Degree of model (eg. linear vs quadratic)
  - Kernels
  - Number of trees
  - Number of layers, filters, batch-size, learning-rate in ANNs

# Hyper-parameters

- Hyper-parameter ≠ parameter (or weights)
  - Parameters are **learned**; hyper-parameters are **chosen**!

- Examples:
  - Degree of model (eg. linear vs quadratic)
  - Kernels
  - Number of trees
  - Number of layers, filters, batch-size, learning-rate in ANNs

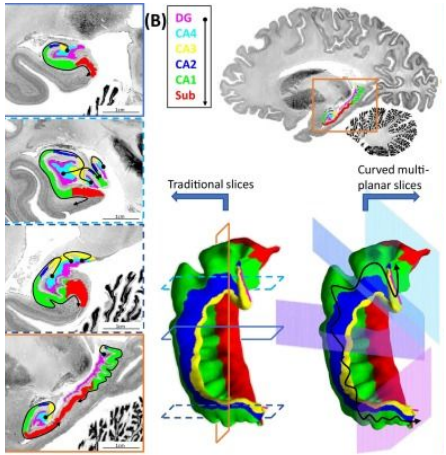
- How do we choose them?
  - Prior beliefs → eg. cortical thickness and age have quadratic relationship.
  - Arbitrarily → we gotta start with something!
  - Trial and error → do a computationally feasible grid-search.

# Pop Quiz

# Model debugging

- **Scenario I: Segmentation of Hippocampus**
  - Human error ~ 5%
  - Train error ~ 20%

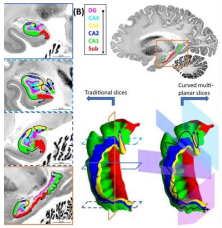
Underfit or Overfit?



# Model debugging

- **Scenario I: Segmentation of Hippocampus**
  - Human error ~ 5%
  - Train error ~ 20%

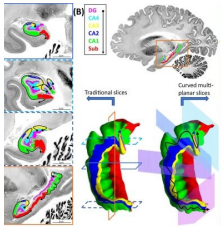
**Underfit** or Overfit? → Bigger/different model



# Model debugging

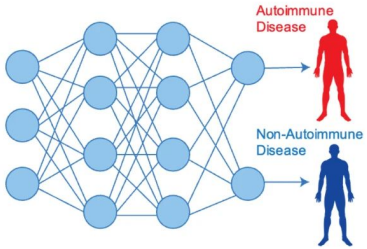
- Scenario I: Segmentation of Hippocampus
  - Human error ~ 5%
  - Train error ~ 20%

  → **Underfit** or Overfit? → Bigger/different model



- **Scenario 2: Medical Dx**
  - Train error ~ 7%
  - Val error ~ 20%

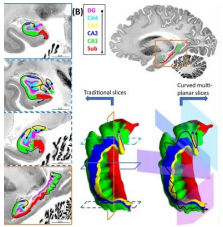
  → Underfit or Overfit?



# Model debugging

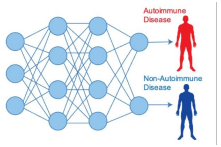
- Scenario I
  - Human error ~ 5%
  - Train error ~ 20%

**Underfit** or Overfit? → Bigger/different model



- **Scenario 2: Medical Dx**
  - Train error ~ 7%
  - Val error ~ 20%

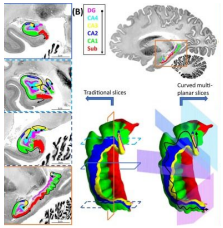
Underfit or **Overfit**? → More data / regularization



# Model debugging

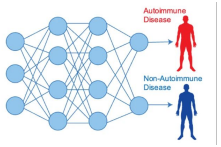
- Scenario I
  - Human error ~ 5%
  - Train error ~ 20%

  → **Underfit** or Overfit? → Bigger/different model



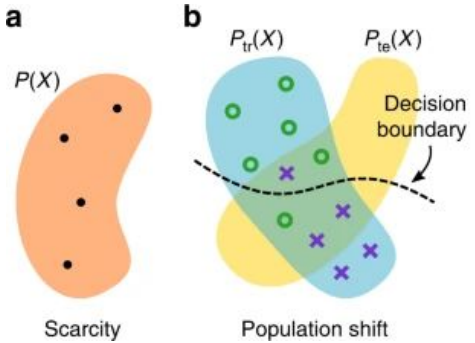
- Scenario 2: Medical Dx
  - Train error ~ 7%
  - Val error ~ 20%

  → Underfit or **Overfit**? → More data / regularization




- **Scenario 3: Clinical trial / replication**
  - CV error ~ 10%
  - New dataset error ~ 25%

  → Underfit or Overfit or Something else?

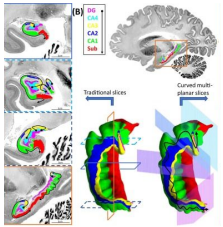



https://www.nature.com/articles/s41467-020-17478-w

# Model debugging

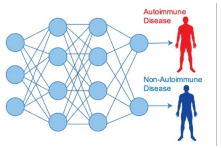
- Scenario I
  - Human error ~ 5%
  - Train error ~ 20%

  → **Underfit** or Overfit? → Bigger/different model



- Scenario 2
  - Train error ~ 7%
  - Val error ~ 20%

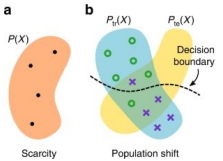
  → Underfit or **Overfit**? → More data / regularization




- **Scenario 3: Clinical trial / replication**
  - CV error ~ 10%
  - New dataset error ~ 25%

  → **Overfit** (feature shift / concept drift)

  → More data + fix labels




https://www.nature.com/articles/s41467-020-17478-w

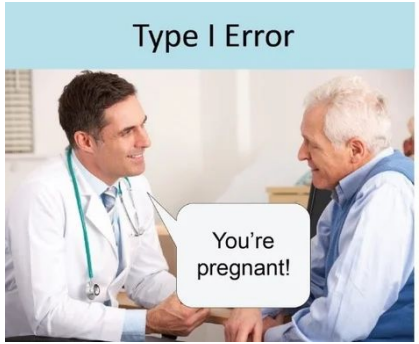
# Performance Scores

- Loss functions → computationally well-suited metrics
  - May / need not completely capture real-life metrics of interest

- Scores → practically useful metrics
  - Binary classification

| Confusion Matrix | | Ground Truth: POSITIVE | Ground Truth: NEGATIVE |
|---|---|---|---|
| Prediction | POSITIVE | TP | FP |
| | NEGATIVE | FN | TN |

**False Positive**




**False Negative**

# Performance Scores

- SZ prediction model (prevalence ~ 1% )
  - FP: model predicts *SZ* when person is *healthy*
  - FN: model predicts *healthy* when person has *SZ*
- What happens if we build model that predicts everyone as healthy?
  - i.e. zero FPs!

# Performance Scores

- SZ prediction model (prevalence ~ 1% )
  - FP: model predicts *SZ* when person is *healthy*
  - FN: model predicts *healthy* when person has *SZ*
- What happens if we build model that predicts everyone as healthy?

| Score | Formula | Null | What does it tell us? | When do I use it? |
|---|---|---|---|---|
| **Accuracy** | (TP+TN) / (TP+FP+FN+TN) | 0.99 | How many people did we correctly predict out of all the people tested? | FNs & FPs have similar costs |
| **Precision (aka PPV)** | TP/(TP+FP) | NaN | How many of those who we predicted as "SZ" do actually have "SZ"? | If you want to be more confident of your TPs |
| **Recall (aka Sensitivity)** | TP/(TP+FN) | 0 | Of all the people who have SZ, how many of those did we correctly predict? | If you prefer FPs over FNs. |
| **Specificity** | TN/(TN+FP) | 1 | Of all the people who are healthy, how many of those did we correctly predict? | If you prefer FNs over FPs. |
| **F1** | 2*(Recall * Precision) / (Recall + Precision) | NaN | Harmonic mean(average) of the precision and recall. | When you have an uneven class distribution |

# Pop Quiz

Goal: Diagnose Schizophrenia (SZ)

Data: MRI + demographics + psychiatric assessments + diagnostic labels

Lit search: Our population has 1% SZ prevalence

We start with splitting data 80% (train) and 20% (test).

**We do a 10 x 3 fold nested CV and find out that our MRI model now has average test accuracy of 91%. Then we also calculate,**

- 90% sensitivity (i.e. probability that prediction is positive if patient has SZ)
- 91% specificity (i.e. probability that prediction is negative if patient doesn't have SZ)

The clinician asks you what are the chances that my patient has SZ, if your model says SZ?

A) 9 in 10 B) 1 in 2 C) 1 in 10 D) 1 in 100

*Also how happy should we be now?*

# Pop Quiz

Goal: Diagnose Schizophrenia (SZ)

Data: MRI + demographics + psychiatric assessments + diagnostic labels

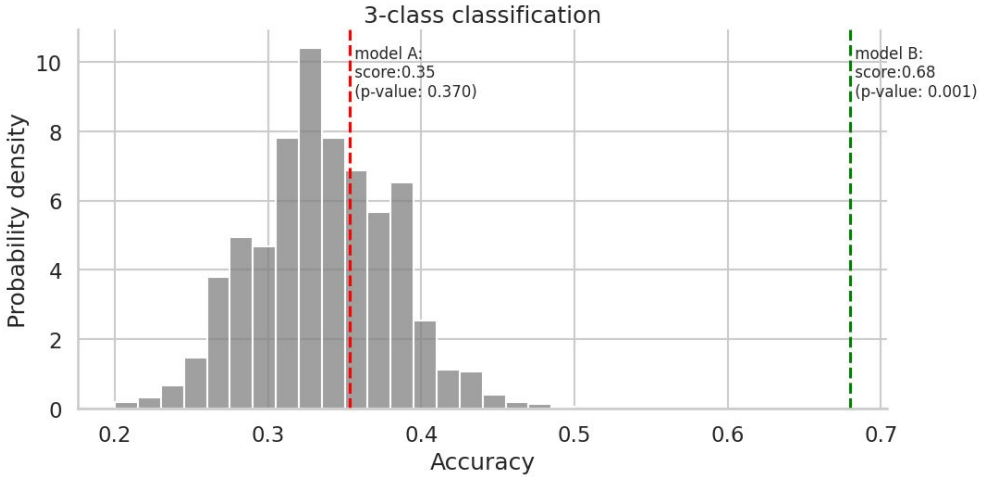
Lit search: Our population has 1% SZ prevalence

**Within the same CV setup, we also train a new model with MRI and demographics and psych-assessments. We now get average test accuracy of 95%.**

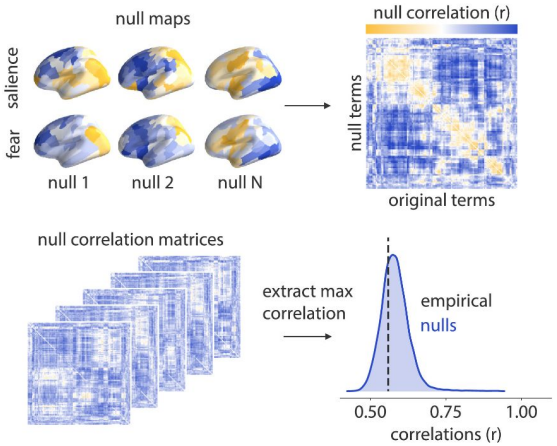
- **Which model is better?**
- **Is the model performance statistically significant?**

# Null performance

- Permutation tests → X remains the same, y is ***randomly*** permuted
  - How to decide “random”?



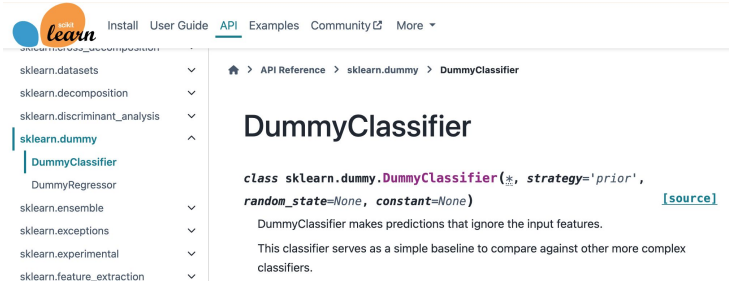

Toy example




Markello & Misic 2021

# Dummy models

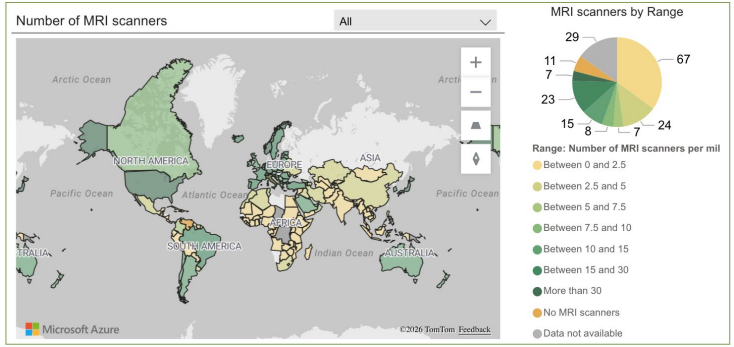
- Sanity checks with “dummy” models
  - Classification: Predict majority class all the time
  - Regression: Predict the median value all the time
- Sanity checks with “simpler” models
  - Try only demographic features
  - Try common heuristics → Does fancy new data modality or model really add predictive value?

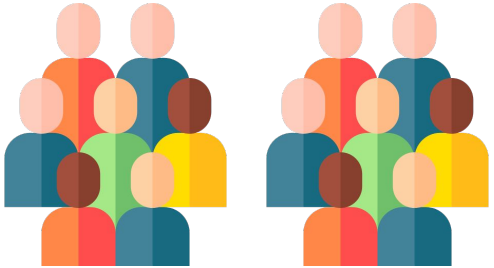


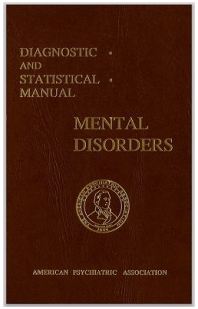
# Beyond the Lab Experiment

# Data Challenges

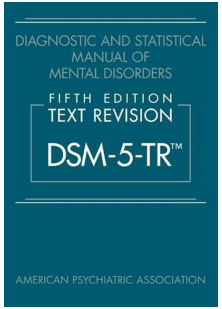
- Population demographics / genetics
- Variability in clinical data collection
- Noisy labels (e.g. Diagnosis definitions)



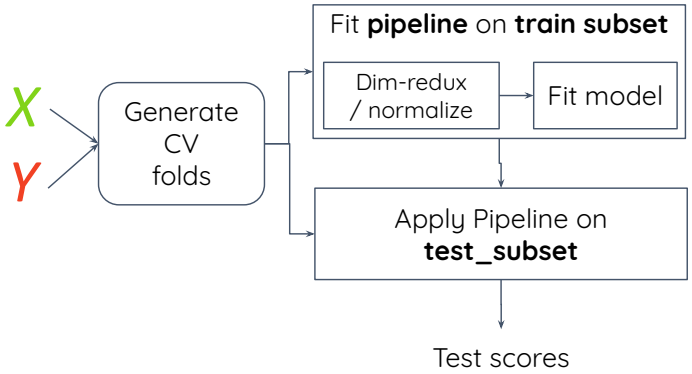


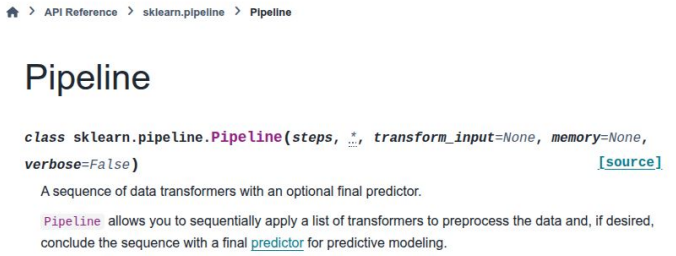

DSM
4 → 5tr



# Methodological Challenges

- Implicit double-dipping
  - Dimensionality reduction → PCA on entire data
  - Data normalization → Z-score on entire data





API Reference › sklearn.pipeline › Pipeline

Pipeline

*class* sklearn.pipeline.**Pipeline**(*steps*, *, *transform_input*=*None*, *memory*=*None*, *verbose*=*False*) [source]

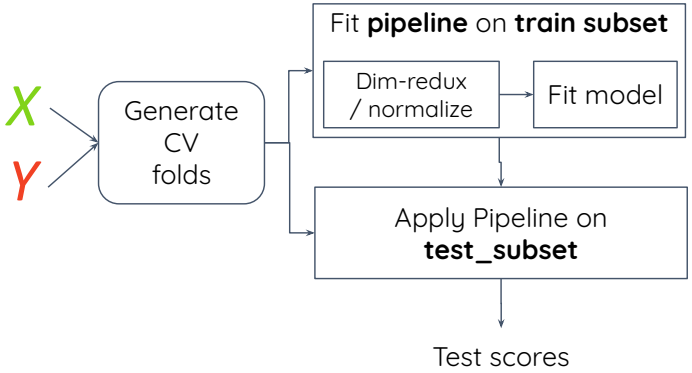
A sequence of data transformers with an optional final predictor.

`Pipeline` allows you to sequentially apply a list of transformers to preprocess the data and, if desired, conclude the sequence with a final predictor for predictive modeling.

https://scikit-learn.org/stable/modules/generated/sklearn.pipeline.Pipeline.html

# Methodological Challenges

- Implicit double-dipping
  - Dimensionality reduction → PCA on entire data
  - Data normalization → Z-score on entire data
- Degrees of freedom before “ML-ready” data
  - Hardware, protocols, pre-processing, post-processing, feature selection
  - Model robustness → stability against input perturbations

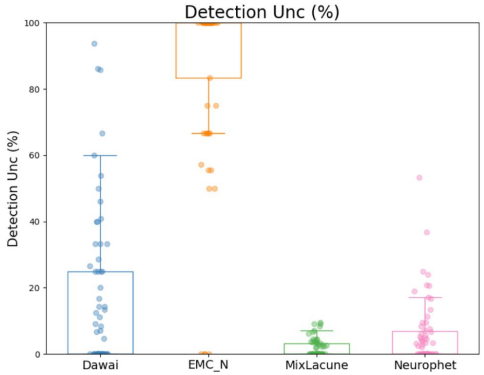


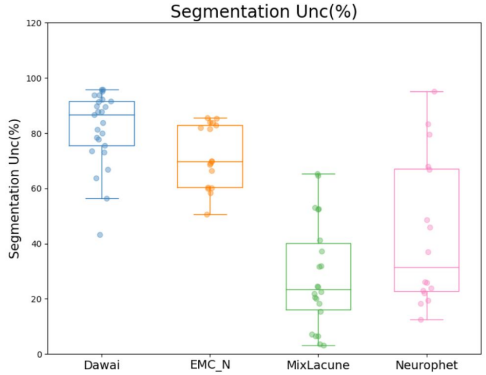

# Model Challenges

- Prediction certainty / confidence
  - A patient is interested in the reliability of a test result in her particular case, ***not in the reliability of the test on average.***
  - Softmax prediction: 70% SZ, 30% control → this is NOT “certainty”
    - Monte Carlo approaches

**MICCAI 2021 challenge*:***
***Where is VALDO?***

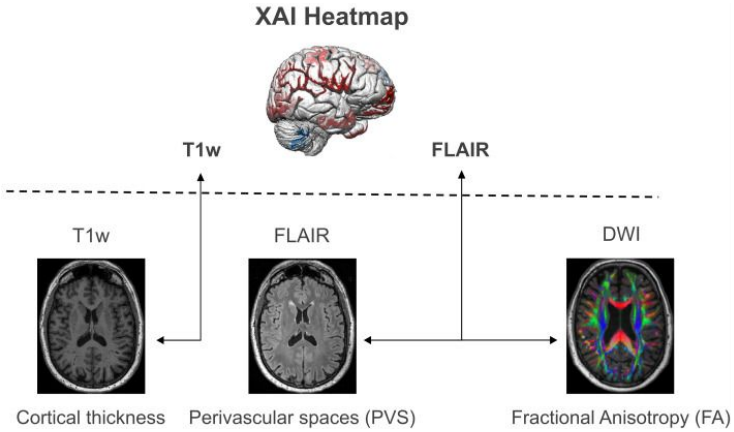
**VAscular Lesions Detection and segmentation**



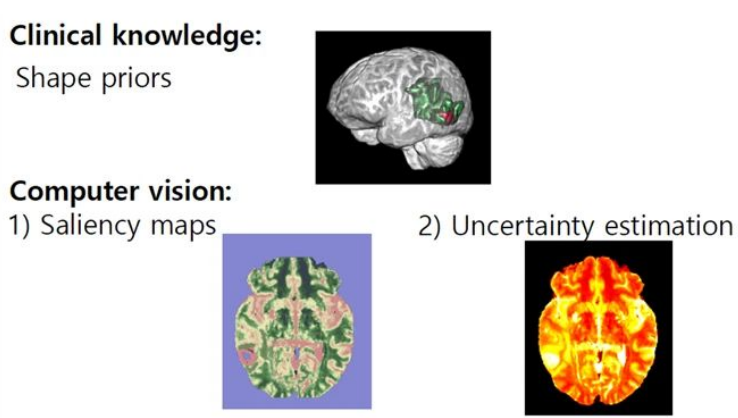



# Model Challenges

- Model explainability / causality
  - Shap: SHapley Additive exPlanations.
  - LIME: Local Interpretable Model-agnostic Explanations
  - Grad CAM: Gradient-weighted Class Activation Map




https://pmc.ncbi.nlm.nih.gov/articles/PMC12319959




https://www.nature.com/articles/s41746-022-00699-2

# ML Novice Checklist

- Data
  - What is my n_features and n_samples?
  - Am I encoding categorical data correctly?
  - Am I using information (e.g. mean) from test set to preprocess (eg. zscore) the data?

# ML Novice Checklist

- Data
  - What is my n_features and n_samples?
  - Am I encoding categorical data correctly?
  - Am I using information (e.g. mean) from test set to preprocess (eg. zscore) the data?

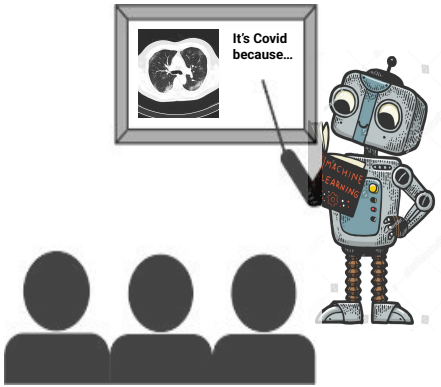
- Model
  - Do my performance metrics capture the practical use-case of interest?
  - What is the null / dummy model performance?
    - Classification: Predict majority class all the time
    - Regression: Predict the median value all the time
  - Am I interpreting model parameters (i.e. weights) correctly?

# Takeaways

- Supervised ML is useful for **predictions** but **not really for explanations**
  - eg. image segmentation, prognosis, drug development

- Our job is to ensure **generalizability** of these models
  - Multitude of validations
  - Understanding model biases and limitations

- ***Engineering tools*** vs *Scientific discovery*
  - Interpretability and explainability




Explainable AI

*Ceci n'est pas un cerveau.*

All *(AI/ML)* models are wrong.
But, some *can be* useful.

# Extra slides

# Useful resources

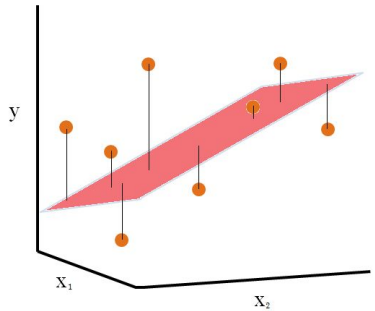
- https://inria.github.io/scikit-learn-mooc/ml_concepts/slides.html
- https://www.3blue1brown.com/topics/linear-algebra
- 3b1b Gradient Descent:
  https://www.youtube.com/watch?v=IHZwWFHWa-w

# Supervised Learning: Models

- Goal: *Learn* parameters (or weights) of a model (M) that maps **X** to **y**

# Supervised Learning: Models

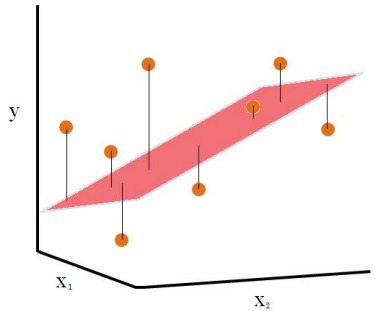
- Goal: *Learn* parameters (or weights) of a model (M) that maps **X** to **y**
- Example models:
  - Linear / Logistic regression



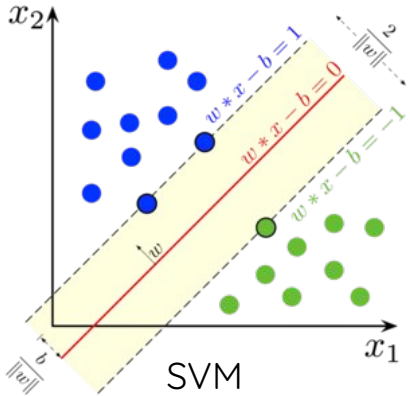

Linear Regression

# Supervised Learning: Models

- Goal: *Learn* parameters (or weights) of a model (M) that maps **X** to **y**
- Example models:
  - Linear / Logistic regression
  - Support vector machines



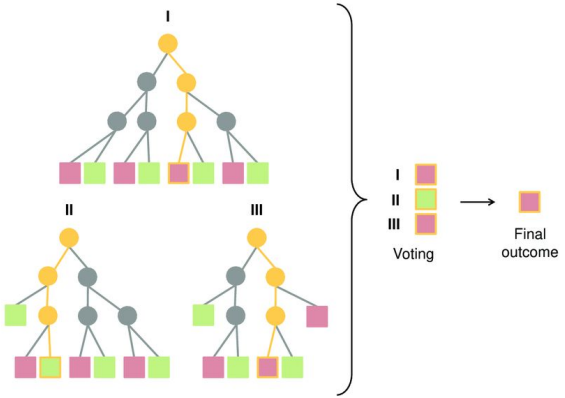

Linear Regression



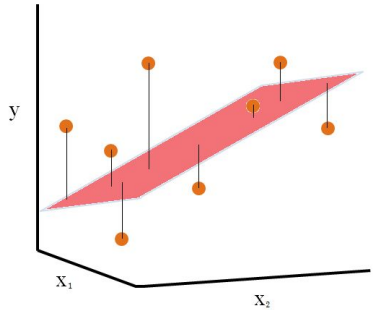

SVM

# Supervised Learning: Models

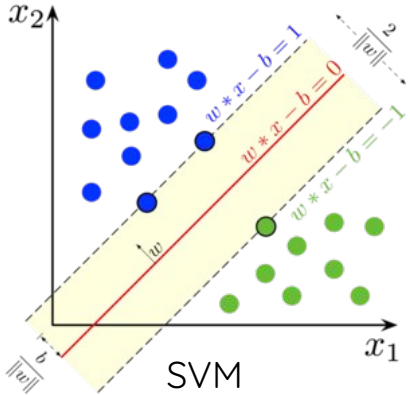
- Goal: *Learn* parameters (or weights) of a model (M) that maps **X** to **y**
- Example models:
  - Linear / Logistic regression
  - Support vector machines
  - Tree-ensembles: random forests, gradient boosting




Tree-ensembles



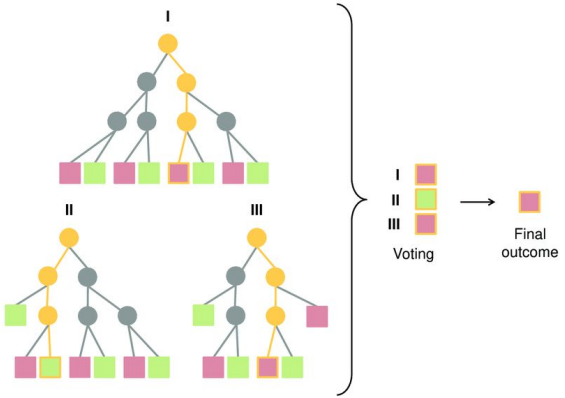

Linear Regression



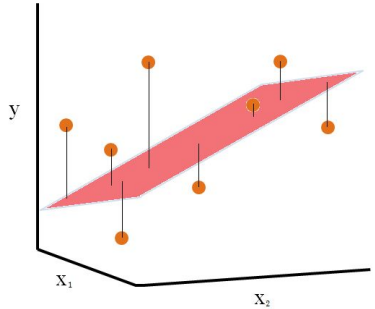

SVM

# Supervised Learning: Models

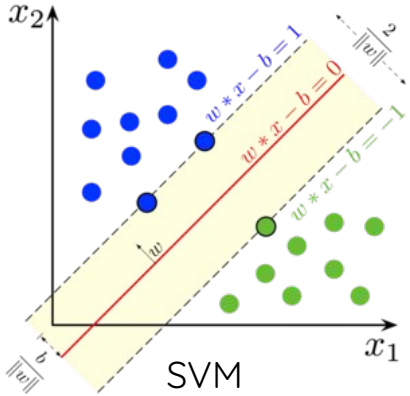
- Goal: *Learn* parameters (or weights) of a model (M) that maps X to y
- Example models:
  - Linear / Logistic regression
  - Support vector machines
  - Tree-ensembles: random forests, gradient boosting
  - Artificial Neural networks



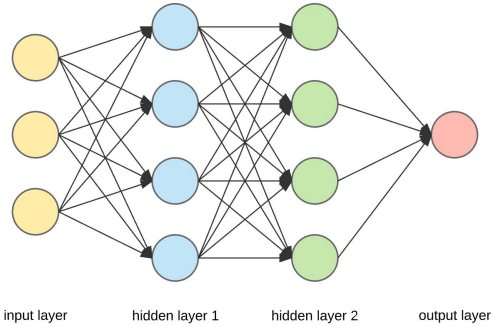

Tree-ensembles




Linear Regression



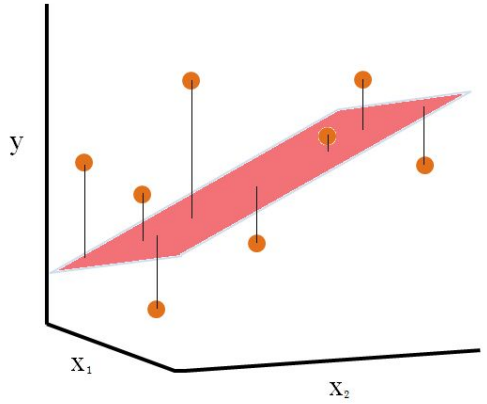

SVM

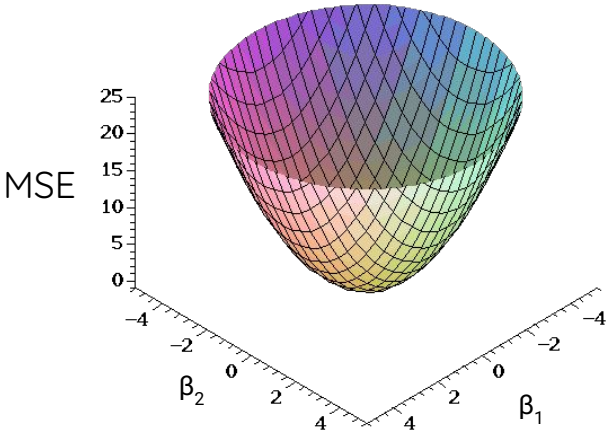



ANN

# Model Fitting

- How do we learn the model weights?
  - Example: Linear regression
  - Model: $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2$
  - Loss function: $MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$
  - Optimization: Gradient descent



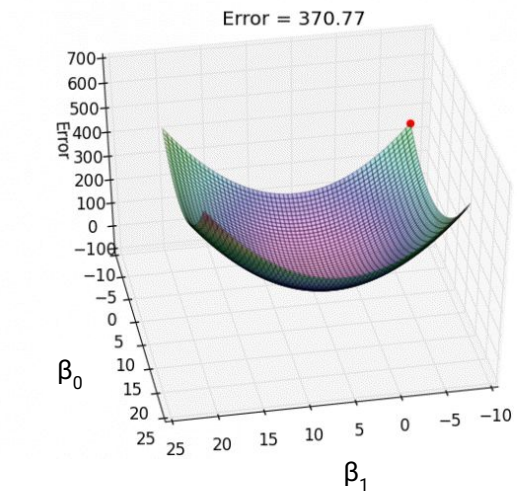



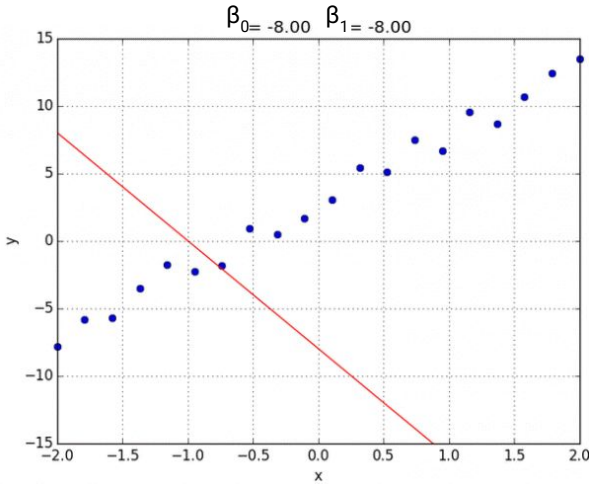
# Model Fitting

- Gradient descent with a **single** input variable and **n** samples
  - Start with random weights ($\beta_0$ and $\beta_1$)
  - Compute loss (i.e. MSE)
  - Update weights based on the gradient

$$\hat{y}_i = \beta_0 + \beta_1 x_i$$

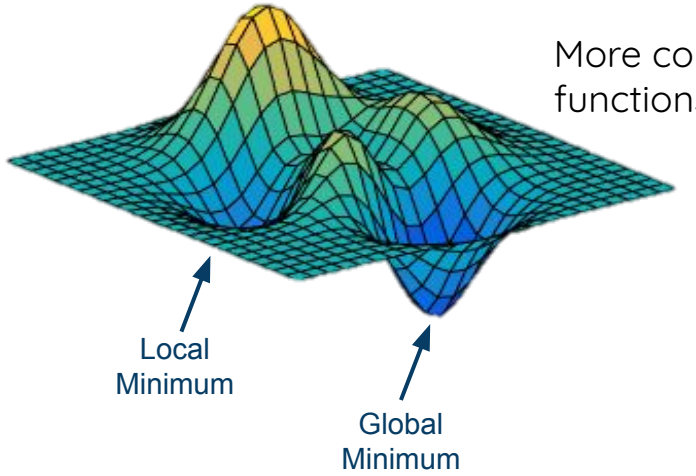
$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$



# Model Fitting

- Gradient descent for complex models with non-convex loss functions
  - Start with random weights ($\beta_0$ and $\beta_1$)
  - Compute loss
  - Update weights based on the gradient




More complex models / loss functions (e.g. ANNs)

# Model Fitting

- Can we control this fitting process to get a model with specific characteristics?

# Model Fitting

- Can we control this fitting process to get a model with specific characteristics?
  - We have strong prior beliefs about what is a **plausible** model
    - e.g. I believe a disease symptom can be predicted with few genes.
  - Practical reasons
    - Prevent overfitting (n_features >> n_samples)

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ldots + \beta_{\rho-1} x_{\rho-1} + \beta_\rho x_\rho$$

# Model Fitting

- Can we control this fitting process to get a model with specific characteristics?
  - We have strong prior beliefs about what is a **plausible** model
    - e.g. I believe a disease symptom can be predicted with few genes.
  - Practical reasons
    - Prevent overfitting (n_features >> n_samples)

- **Yes! → Model regularization**

# Model Fitting: Regularization

- How do we do it?
  - Modify the loss function
  - Constrain the learning process

- Examples:
  - L1 i.e. Lasso
  - L2 i.e. Ridge

# Model Fitting: Regularization

- How do we do it?
  - Modify the loss function
  - Constrain the learning process

- Examples:
  - **L1 i.e. Lasso**
  - **L2 i.e. Ridge**

1) L1/Lasso: constrains parameters to be ***sparse***

$$\text{MSE} = \sum_{i=1}^{n} (y_i - \underbrace{[\beta_0 + \sum_{j=1}^{p} x_{ij}\beta_j]}_{\hat{y}_i})^2 + \underbrace{\lambda \sum_{j=1}^{p} |\beta_j|}_{L_1}$$

2) L2/Ridge: constrains parameters to be ***small***

$$\text{MSE} = \sum_{i=1}^{n} (y_i - \underbrace{[\beta_0 + \sum_{j=1}^{p} x_{ij}\beta_j]}_{\hat{y}_i})^2 + \underbrace{\lambda \sum_{j=1}^{p} \beta_j^2}_{L_2}$$

# Model Fitting: Scikit-learn syntax

```
# import
from sklearn import linear_model, svm


# data
X = [[0, 0], [1, 1]]
y = [0, 1]
```

# Model Fitting: Scikit-learn syntax

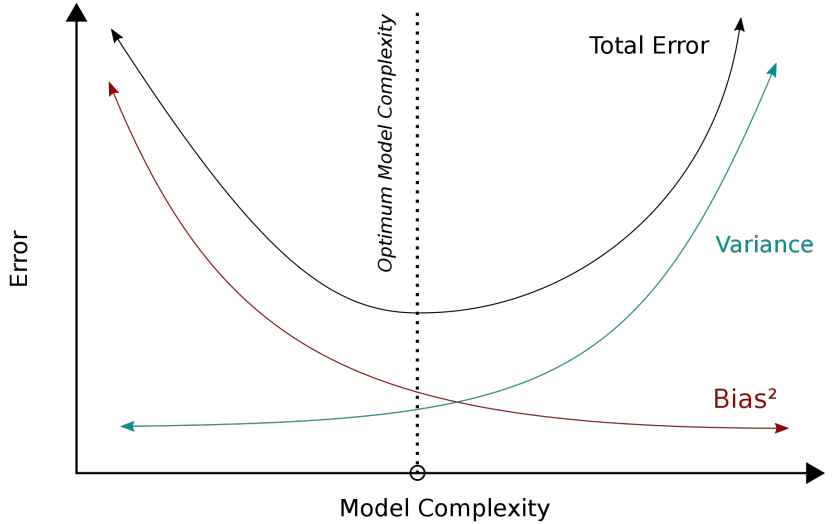
```
# import
from sklearn import linear_model, svm

# data
X = [[0, 0], [1, 1]]
y = [0, 1]

# pick a model
model = linear_model.Lasso(alpha=0.1)

# fit the model with data
model.fit(X, y)
```

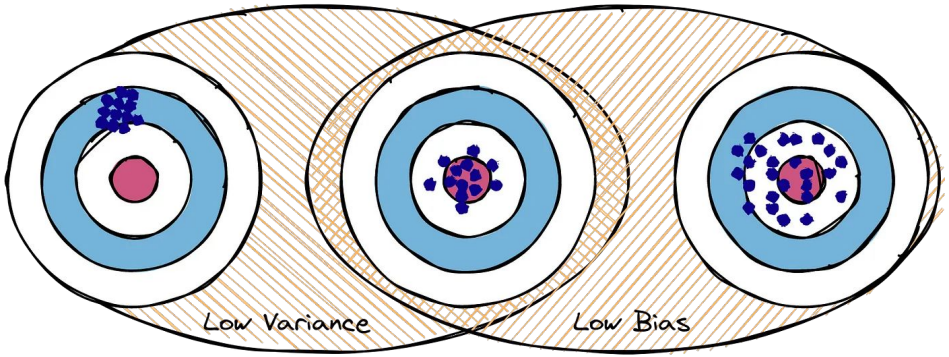
# Model Fitting: Scikit-learn syntax

```
# import
from sklearn import linear_model, svm

# data
X = [[0, 0], [1, 1]]
y = [0, 1]

# pick a model
model = linear_model.Lasso(alpha=0.1)

# fit the model with data
model.fit(X, y)

# predict on new data
y_pred = model.predict([[1, 0]])
```

# Model Evaluation

- Train performance ≠ Test performance
  - Model: Underfitting vs Overfitting
  - Errors: Bias - Variance tradeoff



# Performance Curves

- Receiver Operating Characteristic (ROC) → Want high area-under-the-curve (AUC)
- Precision-Recall → Want high AUC or high Average precision (AP)